

Cấu trúc dữ liệu Prefix Tree - Trie, cài đặt, ứng dụng thuật toán, và nâng cấp

Nguyễn Trần Quang Minh¹

¹Trường Đại học Công nghệ Thông tin, ĐHQG-HCM

Tháng 5 năm 2024

Tóm tắt nội dung

Sau đây là báo cáo trình bày những nghiên cứu về ứng dụng của cấu trúc dữ liệu cây tiền tố hay được gọi là Trie, cùng với những phiên bản nâng cấp của CTDL này. Bên cạnh đó là các cách ứng dụng, cũng như khả năng giải quyết các vấn đề thực tế của các thuật toán này. Và cuối cùng là phần visualization, em xin phép up lên github của report.

Mục lục

1	Quá trình thực hiện	4
1.1	Tuần 1	4
1.2	Tuần 2	4
2	Giới thiệu và mô tả CTDL	4
3	Mô phỏng và cài đặt CTDL Trie	5
3.1	Công tác chuẩn bị và hàm main	5
3.2	Hàm insert	6
3.3	Hàm find	6
3.4	Hàm remove	7
3.5	Đánh giá độ phức tạp	7
3.6	Vận dụng thực tế	7
4	Persistent Trie	7
4.1	Persistent Trie là gì?	7
4.2	Xây dựng thuật toán	8
4.3	Đánh giá độ phức tạp	8
4.4	Vận dụng thực tế	9
5	Aho Corasick	9
5.1	Aho Corasick là gì?	9
5.2	Xây dựng thuật toán	9
5.3	Đánh giá độ phức tạp	10
5.4	Vận dụng thực tế	10
6	Tài liệu tham khảo	10
A	Visualization	11
A.1	Trie	11
A.2	Persistent Trie	11
A.3	Aho Corasick	11
B	Link code + visualization	11

Danh sách hình vẽ

1	Trong một trie, mỗi cạnh được biểu diễn bằng một kí tự, mỗi đỉnh và đường đi từ gốc đến đỉnh đó biểu diễn một xâu gồm các kí tự thuộc các cạnh trên đường đi đó. Ví dụ, đỉnh 5 biểu diễn xâu <i>ab</i> , đỉnh 10 biểu diễn xâu <i>caa</i>	5
2	cây mô phỏng sau khi thêm các xâu	6
3	Hình minh hoạ	8
4	Cây minh hoạ Aho Corasick	10

1 Quá trình thực hiện

1.1 Tuần 1

- Hoàn thành ý tưởng cho bài báo cáo bao gồm 3 phần chính gồm:

1. Mô tả, giải thích, chi tiết cài đặt CTDL Trie.
2. Giới thiệu phiên bản cải tiến của Trie là Persistent Trie.
3. Giới thiệu Aho Corasick có cốt lõi là Trie kết hợp KMP.

- Code xong cả ba thuật toán và upload lên github.

- Viết báo cáo phần đầu tiên về Trie cơ bản, và chỉnh sửa ý tưởng cây Persistent Trie.

1.2 Tuần 2

- Tiếp tục viết báo cáo cho Persistent Trie, chạy thực nghiệm và hoàn thiện.

- Chỉnh sửa ý tưởng, chạy thực nghiệm thuật toán Aho Corasick, vì vốn dĩ đây là một thuật toán khó.

- Thêm video animation mô tả thuật toán insert, delete và find của Trie

2 Giới thiệu và mô tả CTDL

Trie, hay một số tài liệu còn gọi là cây tiền tố, là một cấu trúc dữ liệu dạng cây hữu dụng được dùng để quản lý một tập hợp các xâu. Mặc dù dễ hiểu và dễ cài đặt, trie lại có rất nhiều ứng dụng. Do vậy, trie thường xuyên xuất hiện trong các cuộc thi lập trình ở Việt Nam nói riêng và quốc tế nói chung.

Một trie cơ bản có thể thực hiện ba thao tác sau với độ phức tạp thời gian tuyến tính:

- Thêm một xâu vào tập hợp
- Xóa một xâu khỏi tập hợp
- Kiểm tra một xâu có nằm trong tập hợp đó hay không

Trie là một cấu trúc dữ liệu dạng cây dùng để lưu trữ một danh sách các xâu với bộ ký tự hữu hạn, cho phép việc lưu trữ các xâu hiệu quả có tiền tố giống nhau. Hãy xem xét một ví dụ sau:

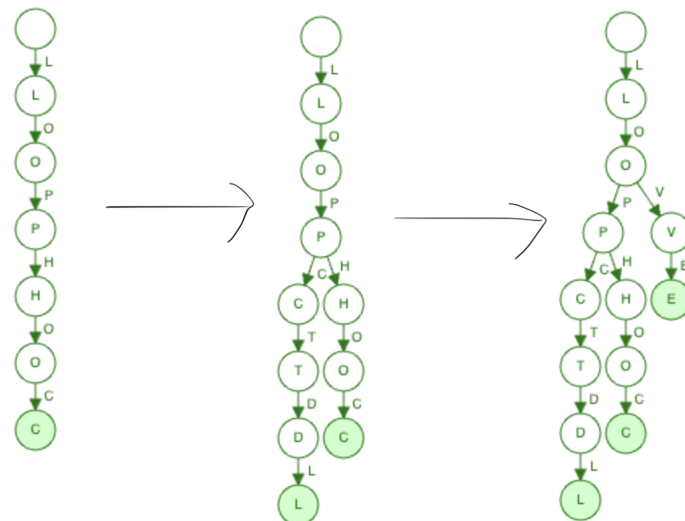
3.2 Hàm insert

Với hàm thêm vào trie, ta bắt đầu tại nút gốc. Ta duyệt qua lần lượt các kí tự trong chuỗi và đi xuống cạnh chứa kí tự tương ứng. Nếu như cạnh tương ứng đó chưa tồn tại thì ta tạo đỉnh mới rồi thêm nó vào mảng *child*.

Sau đây là code C++ sử dụng mảng:

```
1 void insert(string st) {
2     int u = 0;
3     for (auto c : st) {
4         int x = converting(c); // change char to int
5         if (trie[u].child[x] == -1) { // if there is no existence, expand new node
6             trie[u].child[x] = (int)trie.size(); // point to new node location
7             trie.push_back(Node()); // create new node
8         }
9     }
10    trie[u].end_of_word++; // count number of words ending on this node
11 }
```

Ta được mô phỏng sau thêm lần lượt các chuỗi 'lophoc', 'lopctdl', 'love':



Hình 2: cây mô phỏng sau khi thêm các chuỗi

3.3 Hàm find

Hàm tìm chuỗi được cài đặt giống hàm thêm chuỗi. Chỉ khác là nếu không có cạnh tương ứng với kí tự đang duyệt, ta dừng ngay lập tức vì chuỗi đó sẽ không thể xuất hiện trong trie.

Code C++:

```
1 bool check(string st) {
2     int u = 0;
3     for (auto c : st) {
4         int x = converting(c); // convert char to int
5         if (trie[u].next[x] == -1) return false; // return false if next node is empty
6         u = trie[u].next[x]; // go to next node
7     }
8     return trie[u].number_of_word > 0;
9 }
```

3.4 Hàm remove

Ở hàm xoá xâu, ta kết hợp giữa việc kiểm tra xâu đó có tồn tại trong trie hay không và sau đó xoá nó ra khỏi cây. Nếu tồn tại xâu đang cần xoá, chỉ cần giảm biến đếm *numberofword* của node cuối cùng trong quá trình tìm xâu.

Code C++:

```
1 void erase(string st) {
2     int u = 0;
3     for (auto c : st) {
4         int x = converting(c); // convert char to int
5         u = trie[u].next[x]; // go to next node
6     }
7     trie[u].number_of_word--; // decrease counting of word
8 }
```

Sau khi xoá xâu st ra khỏi cây trie, những node đã được thêm mới với xâu st hay đường đi của xâu st trên cây trie vẫn còn tồn tại. Code cần được cải tiến thêm để xoá được những node bị thừa, những node mà không nằm trong bất kỳ xâu con lại nào trong tập xâu.

3.5 Đánh giá độ phức tạp

- Hàm insert
 - Có độ phức tạp thuật toán: cố định là $O(m)$, với m là độ dài của xâu.
 - Có độ phức tạp không gian: nhiều nhất là $O(m * k)$ tương đương $m * k * 4$ bytes bộ nhớ, với m là độ dài của xâu và k là số lượng ký tự.
- Hàm find
 - Có độ phức tạp thuật toán: cố định là $O(m)$, với m là độ dài của xâu.
 - Không có độ phức tạp không gian.
- Hàm remove
 - Có độ phức tạp thuật toán cũng là $O(m)$, với m là độ dài của xâu.
 - Không có độ phức tạp không gian

3.6 Vận dụng thực tế

Dùng Trie để giải quyết các bài toán liên quan đến tìm kiếm xâu con. Trie còn giúp quản lý một hệ cách ký tự một cách trật tự, thống nhất và cực kỳ hiệu quả.

Ở đây, em đã viết một chương trình dùng Trie để quản lý danh sách lớp học, giáo viên có thể thêm sinh viên vào, gỡ bỏ một vài sinh viên, và có thể kiểm tra xem trong danh sách có tồn tại một sinh viên nhất định hay không. Em tin với thuật toán này, khi nâng cấp lên nhiều khía cạnh, thì có thể quản lý nhiều dữ liệu hơn với độ chính xác lớn.

4 Persistent Trie

Là phiên bản nâng cấp của CTDL Trie, nằm trong nhóm dạng cấu trúc nâng cao Persistent đứng cạnh Persistent Segment Tree, Persistent Disjoint Set và Persistent Fenwick Tree.

4.1 Persistent Trie là gì?

Trước hết, Persistent Data Structures là những cấu trúc dữ liệu được dùng khi chúng ta cần có toàn bộ lịch sử của các thay đổi trên 1 cấu trúc dữ liệu.

Với Persistent Trie, ctld nâng cấp này cho phép truy cập vào cây trie trong bất kỳ thời điểm nào trong lịch sử các thay đổi về cấu trúc cây.

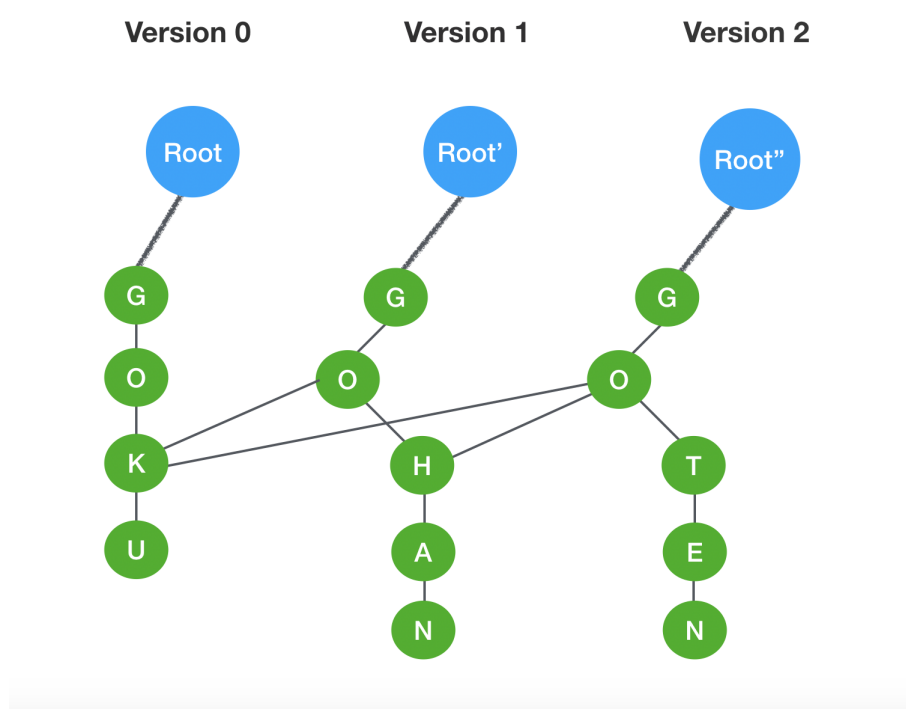
4.2 Xây dựng thuật toán

Công việc của chúng ta là áp dụng kiến trúc về Persistency vào ctdl Trie sẵn có, mà vẫn đảm bảo tốc độ tính toán cũng như khối lượng bộ nhớ.

Cùng suy nghĩ về các phiên bản, tức là với mỗi lượt thay đổi/chèn vào Trie, chúng tôi sẽ tạo một phiên bản mới của nó. Ta ghi nhận phiên bản khởi đầu là $version_0$. Từ đây, bất cứ lúc nào ta thêm xâu, ta sẽ tạo thêm một version hoàn toàn mới toanh có thể gọi là $version_{n+1}$ từ phiên bản trước đó là $version_n$.

Nhưng việc tạo toàn bộ Trie mới mỗi lần cho mọi phiên bản sẽ khiến bộ nhớ tăng gấp đôi và ảnh hưởng rất xấu đến độ phức tạp không gian bộ nhớ. Vì vậy, ý tưởng này sẽ dễ dàng hết bộ nhớ đối với một số lượng lớn các phiên bản.

Cùng khai thác đặc điểm là với mỗi lần chèn mới vào trie, chính xác các nút X (length-of-key) sẽ được truy cập/sửa đổi. Vì vậy, phiên bản mới của chúng ta sẽ chỉ chứa X nút mới này và các nút trie còn lại sẽ giống như phiên bản trước. Do đó, khá rõ ràng rằng đối với mỗi phiên bản mới, chúng ta chỉ cần tạo X nút mới này trong khi các nút trie còn lại có thể được chia sẻ từ phiên bản trước.



Hình 3: Hình minh họa

Giờ thì câu hỏi được đặt ra: Làm sao để theo dõi, cũng như quản lý tất cả các phiên bản?

Chúng ta chỉ cần theo dõi nút gốc của version mới nhất được thêm vào cây Trie, điều này sẽ nhằm mục đích theo dõi tất cả các nút mới được tạo trong các phiên bản khác nhau vì nút gốc mới nhất cung cấp cho chúng ta điểm vào cho các phiên bản cụ thể trước đó. Với cơ sở này, chúng ta có thể duy trì một mảng các con trỏ tới nút gốc của trie cho tất cả các phiên bản.

Cài đặt: Các bạn có thể xem cài đặt Code C++ của mình ở link github.

4.3 Đánh giá độ phức tạp

Mình đã nhắc sơ qua phần tính toán độ phức tạp để tối ưu đến mức tối thiểu. Nhìn chung, thuật toán chỉ mang độ phức tạp tuyến tính $O(n * k)$, với n là số ký tự được thêm vào hệ cây và k là số lượng ký tự khác nhau. Về độ phức tạp bộ nhớ, thuật toán sẽ chiếm nhiều nhất $O(n * k)$, tương đương với $n * k * 4$ bytes bộ nhớ, với n là số ký tự được thêm trong cây và k là số lượng ký tự khác nhau.

4.4 Vận dụng thực tế

Ngoài các chức năng cơ bản, biểu tượng cũng như độ nhất của Trie, Persistent Trie cho phép người dùng truy cập vào một mốc thời điểm nào đó trong quá khứ để tiến hành tìm kiếm các dữ liệu.

Do đó, trong thực tế, chức năng tìm kiếm sinh viên sẽ được thêm một tính năng nâng cấp, một điều kiện chính là từ khoảng thời điểm nào đó trở về quá khứ, liệu sinh viên A có nằm trong danh sách W không.

5 Aho Corasick

5.1 Aho Corasick là gì?

Như các bạn đã biết:

- Thuật toán KMP giúp tìm kiếm 1 xâu con (pattern) trong 1 xâu lớn với độ phức tạp $O(N + M)$ với N và M là độ dài 2 xâu.
- Cấu trúc dữ liệu Trie giúp chúng ta lưu trữ và tìm kiếm trong $O(n)$.

Aho Corasick là thuật toán kết hợp giữa trie và kỹ thuật so khớp xâu KMP.

Thuật toán Aho Corasick cho phép chúng ta tìm kiếm nhanh chóng nhiều mẫu trong một văn bản. Tập hợp các chuỗi mẫu còn được gọi là từ điển. Ta sẽ biểu thị tổng độ dài các chuỗi cấu thành của nó là m và kích thước của bảng chữ cái bằng k . Thuật toán xây dựng một automaton hữu hạn trạng thái dựa trên trie trong $O(mk)$ thời gian và dùng nó để xử lý văn bản.

Aho Corasick là một thuật toán khó, cụ thể là khó hiểu, khó nhớ, khó dùng và rất khó cài đặt. Vì thế mình chỉ giới thiệu sơ qua về thuật toán này. Code C++ của mình ở trong link github.

5.2 Xây dựng thuật toán

Đúng vậy, điều chúng ta cần làm là trên mỗi node của cây trie, tạo ra thêm một biến so khớp kmp. Trên thực tế, khi ta có được CTDL Trie, ta cần phải xoay sở để thêm vào nó biến fail trên mỗi node của cây để nếu cây không còn khớp tiếp xâu được nữa thì nó vẫn trở nhanh về node có độ khớp dài nhất. Đó được gọi là suffix link (kết nối tiền tố). Chi tiết xây dựng gồm các bước sau:

- Dựng cây Trie trên các xâu đã cho trước. Với mỗi nút khởi tạo $fail = 0, next[c] = -1$ với mọi c , $matches$ là danh sách rỗng. Sau khi thêm xâu s_i , ta thêm phần tử i vào $matches$ của nút kết thúc xâu.
- Với mảng $next$ trên mỗi node của Trie, $next[c]$ sẽ trỏ đến node khớp dài nhất trong hệ xâu cho trước. Tức là, giả sử tiền tố tại nút x là "minh", và không tồn tại trong tiền tố "minh" + c trong hệ xâu, thì $trie[x].next[c]$ sẽ trỏ đến nút sâu nhất sao cho tiền tố tại $trie[x].next[c]$ là hậu tố của nút x .
- Duyệt BFS trên cây Trie để tính $fail$ và $next$ cho mỗi nút

Sau đây là struct quản lý 1 nút của cây trie trong Aho Corasick(Code C++):

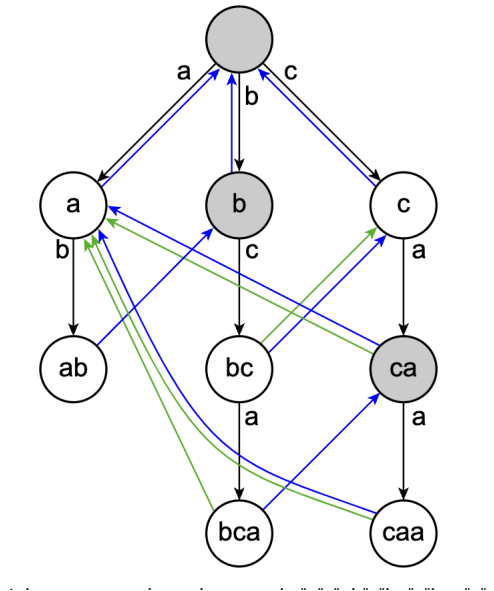
```
1 struct Node {
2     int next[ALPHABET];
3     int fail; // kmp at this node
4     int len; // depth level
5     vector<int> matches; // list nodes' id
6     Node() {
7         REP(i, ALPHABET) next[i] = -1; // point to null
8         fail = 0; // all kmp = 0
9         len = 0; // still dont have level yet
10        matches.clear();
11    }
12};
```

Sau đây là hàm main của Aho Corasick(Code C++):

```

1  vector<Node> trie;
2  int main() {
3      cin >> n; // input number of node
4      trie.push_back(Node()); // initialization root
5      FOR(i, 1, n) {
6          string st; cin >> st;
7          add(st, i); // add string to trie
8      }
9      AC_BFS(); // run bfs
10 }

```



Hình 4: Cây minh hoạ Aho Corasick

- Mũi tên đen thể hiện cây Trie
- Mũi tên xanh lam trở vào cho biến *fail* của nút, tức biến khớp sâu của nút
- Mũi tên xanh lá cây trở vào nút *fail* của nút *fail* của nút *x* (tức $trie[trie[x].fail].fail$)

5.3 Đánh giá độ phức tạp

Với n là số lượng ký tự được thêm vào, k là số ký tự khác nhau, độ phức tạp của thuật toán chỉ nằm ở mức $O(n * k)$ với thao tác BFS toàn cây. Thuật toán chiếm $O(n * k)$ bộ nhớ với $n * k * 4$ bytes bộ nhớ.

5.4 Vận dụng thực tế

Với phiên bản cực mạnh Aho Corasick này, con người có thể vận dụng nó để tối ưu thuật toán tìm kiếm chuỗi/chuỗi để thực hiện tốt vai trò tối ưu hệ thống, làm cho hệ thống chạy được nhanh hơn, chưa được nhiều dữ liệu hơn.

Trên thực tế, Aho Corasick Automation đã được áp dụng, thực thi trên các phương tiện tìm kiếm, cũng như trên các từ điển đại chúng như <https://dictionary.cambridge.org> hoặc <https://www.oxfordlearnersdictionaries.com>

6 Tài liệu tham khảo

Một số tài liệu tham khảo mà các bạn có thể xem thêm về Prefix Tree:

- VNOI
- Geeksforgeeks
- Video hướng dẫn
- Video giải thích(bằng tiếng Anh)

A Visualization

A.1 Trie

A.2 Persistent Trie

A.3 Aho Corasick

B Link code + visualization

<https://github.com/quangminh0604/final-dsa-project>